

Sistemas Distribuídos e Computação em Nuvem

Aula 14 — Orquestração, serverless e computação na borda

C3 · Nuvem, implantação e segurança · 12/11/2026

Prof. M.Sc. Howard Cruz Roatti · FAESA · 2026/2

Onde estamos — C3

C1 (1–7)  · C2 (8–12) 

C3 · **Nuvem, implantação e segurança** (13–18)

Nuvem/containers  · **orquestração/serverless** ·
observabilidade · segurança.

 Três ideias: **orquestração** (coordenar vários containers), **serverless** (entregar só a função) e **borda** (processar perto do usuário).

 **Aula 14.** Você já tem containers soltos; hoje eles viram um **sistema** que sobe com **um comando**.

Retomada

 Você publicou o serviço (URL pública) e containerizou os serviços do C3.A2?

- Cada serviço tem seu container — mas subir **4 containers na mão**, na ordem certa, com a rede certa, é **inviável**.
- Hoje: **um comando** sobe a stack inteira, e vemos alternativas (**serverless**, **borda**) para partes do sistema.

Objetivos desta aula

Ao final, você será capaz de:

1. **Orquestrar** múltiplos containers com **docker-compose** (subir e **escalar** com um comando).
2. **Explicar** o modelo **serverless (FaaS)** e seu preço (**cold start**, limites, sem estado).
3. **Reconhecer** quando a **computação na borda (edge)** reduz latência.

Orquestração: containers isolados não são um sistema

Um serviço precisa **achar** o outro, subir na **ordem** certa, compartilhar uma **rede**. O **docker-compose** descreve tudo num arquivo e sobe com **um comando**.



💡 **Em miúdos:** o compose é a **planta do condomínio**: diz quais prédios existem, como se ligam e quem tem portão para a rua. Um comando ergue tudo.

Escalar com um comando

- Precisa de mais capacidade num serviço? `docker compose up --scale inferencia=3` → 3 réplicas.
- O gateway distribui entre elas; você **vê** réplicas diferentes respondendo (o `host` muda).
- Em **produção**, o orquestrador padrão é o **Kubernetes**: faz isso em **vários servidores**, reinicia o que cai e escala sozinho (noção — não é foco da disciplina).

📌 Orquestração resolve o "como rodar **muitos** containers juntos, de forma confiável". Compose no laboratório; Kubernetes no mundo de produção.

Serverless (FaaS): você entrega a função

Você **não** gerencia servidor: envia **uma função**, e a plataforma a **executa sob demanda**, criando **uma instância por requisição** e **escalando a zero** quando ociosa.



💡 **Em miúdos:** é a **luz com sensor de presença** — acende quando alguém chega, apaga sozinha. Você paga só os segundos que a função rodou.

O preço do serverless

- **Cold start:** se a função estava "apagada", a **1ª** invocação demora mais (subir o ambiente). Já vimos esse nome na Aula 6.
- **Limites:** tempo máximo de execução, memória e tamanho — **não** serve para tarefas longas.
- **Sem estado:** cada invocação é **independente**; guarde estado **fora** (banco, fila, cache).

⚠ Serverless brilha em cargas **esporádicas** e **curtas** (um webhook, um redimensionar imagem). Para um serviço **sempre quente** e de alto volume, um container dedicado costuma sair melhor.

Computação na borda (edge)

Em vez de ir até um datacenter **distante**, parte do processamento acontece **perto do usuário** (CDN, pontos de presença, dispositivos IoT).



💡 **Em miúdos:** é a **farmácia do bairro** em vez do **centro de distribuição**: o que é comum fica pertinho (rápido); o resto vem de longe só quando preciso.

Laboratório

Subir a stack com um comando + publicar uma função serverless

Mão na massa — siga o Guia do Laboratório

Hoje são **4 missões**: subir a stack com **um comando**, **escalar** um serviço, **publicar uma função serverless** (com 3 medições de tempo) e **compor a stack do C3.A2**.

👉 **Abrir o Guia do Laboratório »**

no índice do site: Aula 14 → [Guia do Lab](#)

💡 Exemplo pronto e testado: `exemplos/aula14/` — `gateway/` + `inferencia/` + `docker-compose.yml` (stack) e `funcao/handler.py` (serverless).

O que o exemplo mostra (aquecimento)

```
docker compose up --build          # sobe gateway + inferencia juntos
curl -X POST localhost:8000/infer -d '{"texto\":"gostei, otimo\"}'
# {"via":"gateway","resultado":{"sentimento":"positivo","host":"<id-do-container>"}}
docker compose up --scale inferencia=3  # 3 replicas; o "host" varia nas respostas

python funcao/handler.py           # a funcao serverless, rodando "sob demanda"
```

💡 O gateway acha a inferência pelo **nome do serviço** (`http://inferencia:8000`) na rede do compose — sem IP fixo.

No seu trabalho — C3.A2

- **Componha** a stack inteira do projeto num `docker-compose.yml` : gateway + inferência + worker + armazenamento vetorial, subindo com **um comando**.
- **Integre a borda**: identifique uma parte que ganharia com **edge/serverless** (ex.: um pré-processamento curto ou um webhook) e descreva/implemente.

💡 Puxa para o kit: só o **gateway** expõe porta; os demais são internos e se acham pelo nome. É o padrão de hoje aplicado ao seu projeto.

Atividade para casa

1. **Componha** o `docker-compose.yml` do **C3.A2** e suba o projeto **inteiro com um comando**.
2. **Publique** uma **função serverless** (um componente simples do projeto) e registre **3 medições** de tempo de resposta (incluindo a **1ª**, para ver o cold start).
3. **Integre** o componente serverless/edge ao fluxo do projeto.

📌 **Entregar até a próxima aula:** o projeto subindo com um comando + a função serverless publicada com as ****3 medições****. Roteiro no **Guia do Laboratório**.

◆ Foco ENADE

O que costuma cair:

- **Orquestração** (compose/Kubernetes): coordenar e **escalar** containers.
- **Serverless/FaaS**: sob demanda, escala a zero, **cold start**, **sem estado**.
- Quando usar serverless × container dedicado (esporádico/curto × sempre quente).
- **Edge computing**: processar perto do usuário para **reduzir latência**.

Termos-chave: Orquestração · docker-compose · Kubernetes · Serverless/FaaS · Cold start · Edge

💡 Pegadinha: serverless "nunca tem custo de latência". Tem: o **cold start** da 1ª invocação após ociosidade.

Questão de autoavaliação (estilo ENADE)

Uma função **serverless** que ficou **ociosa** recebe uma requisição e demora bem mais para responder **nessa primeira** chamada; as seguintes são rápidas. Esse atraso inicial é explicado por:

- A) uma falha de rede intermitente
- B) **o cold start: a plataforma precisa inicializar o ambiente da função antes de executá-la**
- C) o fato de funções serverless serem sempre lentas
- D) a ausência de internet no datacenter
- E) um erro de idempotência

Resolução — alternativa B

- **Cold start:** quando a função está "apagada" (escalou a zero), a plataforma **sobe o ambiente** antes de rodar → a 1ª chamada paga esse custo.
- As seguintes reaproveitam a instância "quente" → rápidas. Não é rede (A/D) nem idempotência (E).

💡 Mesmo conceito da Aula 6 (carregar o modelo). Serverless troca custo de servidor ocioso por cold start ocasional.

Fora da sala · Glossário

Para estudar

- Docs do **Docker Compose** e, por curiosidade, do **Kubernetes** (conceitos).
- Docs de **AWS Lambda / Cloud Functions** (o que é um handler).
- Rode o exemplo com `--scale` e veja o `host` variar.

Glossário

- **Orquestração:** coordenar vários containers como um sistema.
- **docker-compose:** descreve e sobe a stack local.
- **Kubernetes:** orquestrador de produção (vários servidores).
- **Serverless/FaaS:** você entrega a função; a plataforma executa sob demanda.
- **Cold start:** atraso da 1ª invocação após ociosidade.
- **Edge:** processar perto do usuário.

Até a próxima aula

Suba o C3.A2 com um comando e publique a função serverless (3 medições).

Próxima (Aula 15): Observabilidade — quando algo fica lento numa stack de vários serviços, como **enxergar** onde está o gargalo (logs, métricas e **tracing**).

← Anterior

Aula 13 · Nuvem/containers

≡ Índice

todas as aulas